FSD
Semester 1, 2026

Week 09

dipto.pratyaksa@rmit.edu.au

Full stack- ***part 3***

More on TypeORM

GraphQL- part 1;

Unit Testing

RMIT UNIVERSITY

Official CES (course experience survey)

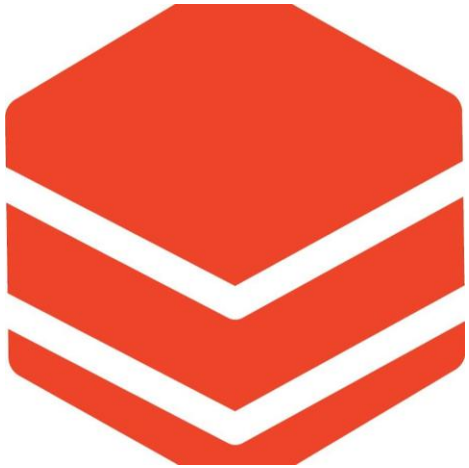
- It is that time again and we really need your feedback about this course. It helps us improve the course and experience for other students and you get your voice heard.
- Please finish the CES survey for this course via this anonymous feedback form. You can access the survey via the URL:
  - https://surveys.rmit.edu.au/blue
- It only takes few minutes, please do not leave it till the last minute.

Segment 1

Full stack development – *part 3*

More on database programming

ORM (TypeORM)



Database Tools

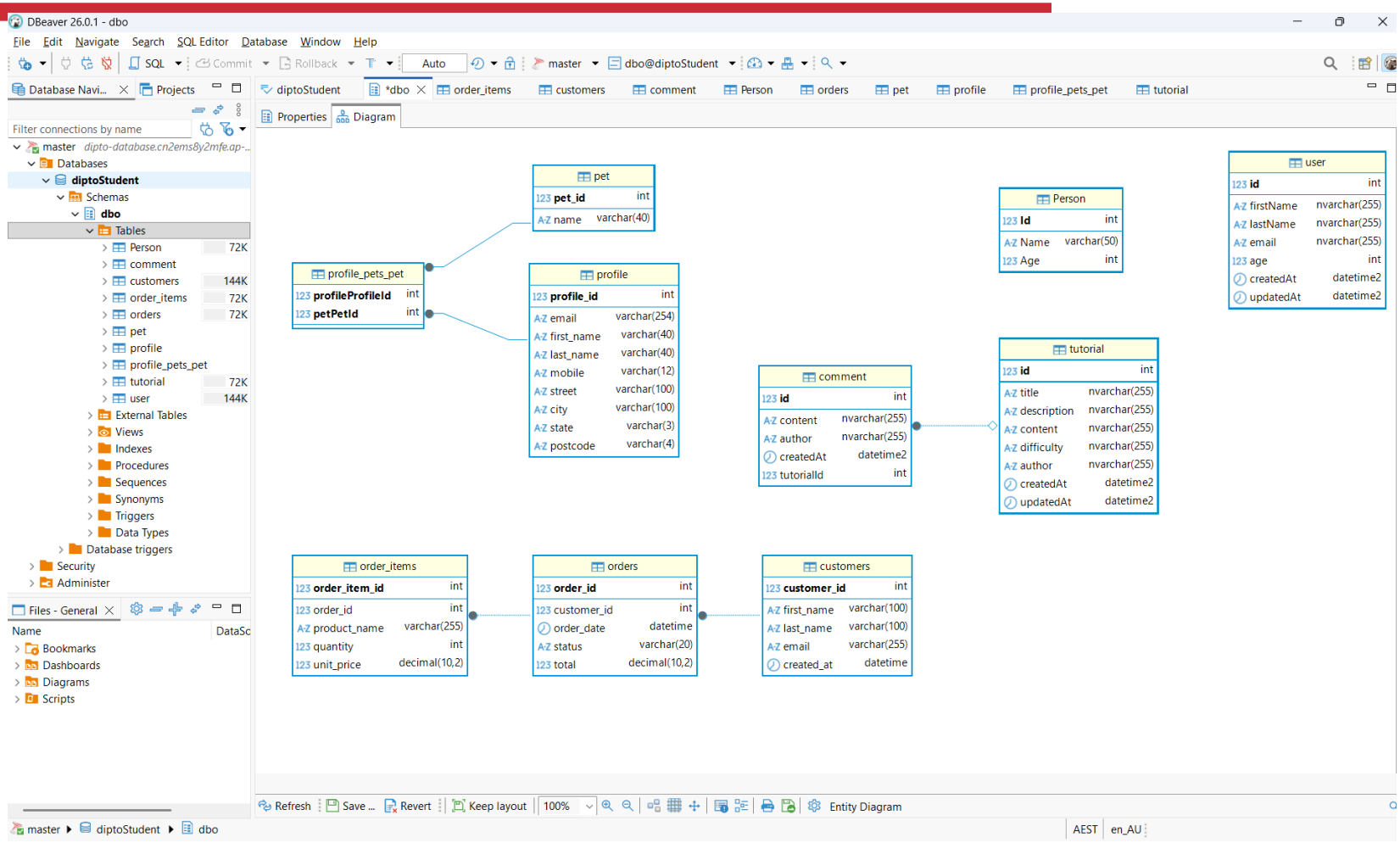
DBeaver: Open-Source DBMS client to administer your DB tables and draw Entity Relationship Diagram

VS Code mssql extension: SQL Server viewer / editor integrated with your code editor. Good for quick look up.

SSMS: SQL Server Management Studio to help reset password

RMIT

DBeaver



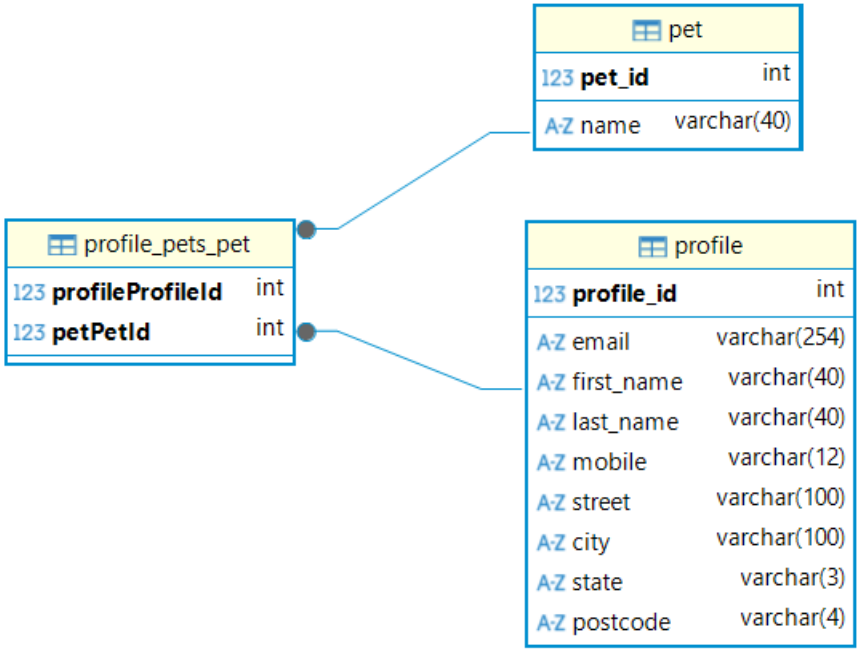
RMIT

Database programming

- In week 7's lectorial- we covered a big example of a full stack app which involved creating a CRUD application around a table called 'users'
- In week 8's lectorial- we covered some examples
  - Two around a tables called tutorial, comment
- Now we will look at an example which will involve creating a full stack app around
  - Editing profile
  - Many to many relationship

example1: node+express+TypeORM

- **The ERD (*entity relationship diagram*) for the scenario:**



example1: node+express+TypeORM

- Observe the many-to-many relationship between profile and pet.
- A profile can have many pets and a pet can belong to many profiles.
- *Many to many (M:N)* relationship is set inside entity/Pet.ts file and entity/Profile.ts
- We can then fetch all profiles connected to a pet and all pets connected to a profile
- Remember eager fetching from last week, let's review that again

TypeORM: multiple features

- Let us talk about the following features:
  1. Database constraints
  2. Raw queries
  3. Migrations

TypeORM constraints

Length, data type and nullable or not

```
@Column({ type: "varchar", length: 4, nullable: true })
postcode: string;
```

Uniqueness

```
@Column({ type: "varchar", length: 254, unique: true })
email: string;
```

Validation using class-validator

- We can use decorators similar to our @Column etc to do validation on our inputs
- Add class-validator and class-transformer packages to your project
- Add the Express middleware
- Create your DTOs
- Use them in your routes

```
import { IsEmail, IsNotEmpty, IsString } from "class-validator";

export class CreateProfileDTO {
  @IsEmail()
  email: string;

  @IsString()
  @IsNotEmpty()
  first_name: string;

  @IsString()
  @IsNotEmpty()
  last_name: string;
}
```

```
router.post("/", validateDto(CreateProfileDTO), (req, res) =>
  profileController.create(req, res)
);
```

Validation using class-validator

- This will then properly validate your inputs using Javascript

Body Cookies Headers (8) Test Results

{} JSON Preview Visualize


```
{
    "errors": [
        {
            "property": "email",
            "constraints": {
                "isEmail": "email must be an email"
            }
        },
        {
            "property": "first_name",
            "constraints": {
                "isNotEmpty": "first_name should not be empty",
                "isString": "first_name must be a string"
            }
        },
        {
            "property": "last_name",
            "constraints": {
                "isNotEmpty": "last_name should not be empty",
                "isString": "last_name must be a string"
            }
        }
    ]
}
```

TypeORM: raw queries

- TypeORM the database query may be a complex one
- TypeORM may not have an equivalent way to write such a query
  - Remember: we talked about ORMs not being perfect 100% replacements of SQL
- In such cases, you must embed a *Raw SQL* query in your controller file (where you are writing the logic of REST API methods)
- You could do something as

```
this.profileRepo.query(`SELECT avg(age) FROM profile`)
```

- This will not be needed for your assessment however you never know- some project you are working on in future may require this feature

Lectorial Exercise

- Complexity aside, what could be the other reasons for embedding a raw SQL query in a TypeORM controller?

TypeORM: migrations

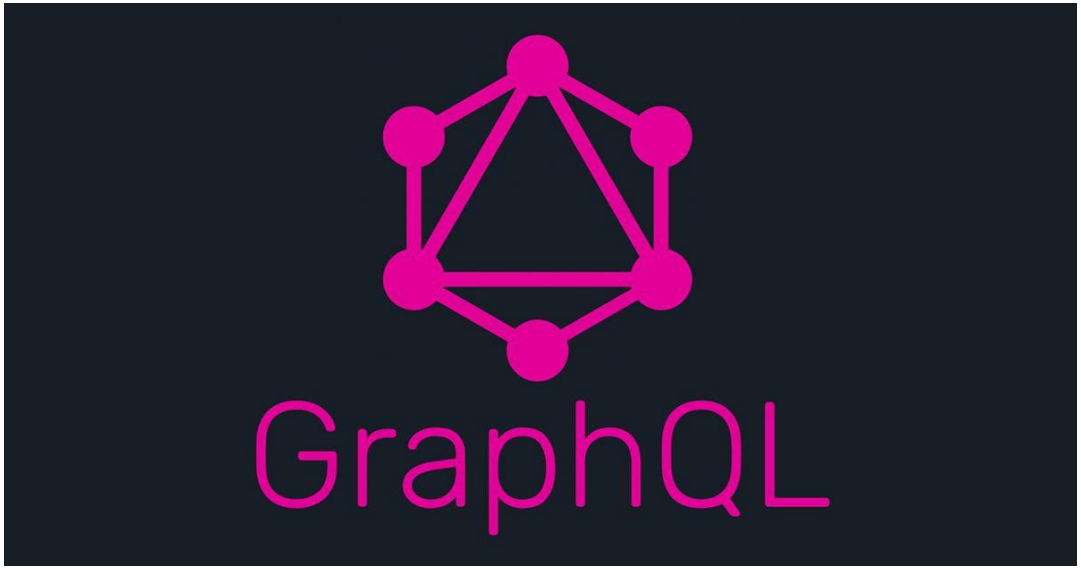
- Migrations are used to modify database structure
- A migration feature is used to keep track of the changes to your database in a Typescript / JS file
- By using migrations, you will be able to save instructions that change your database structures and definitions.
- A migration file has two methods
  1. up(): defines the table name and its columns
  2. down(): drops the table to undo what the up() function has done

TypeORM: migrations

https://orkhan.gitbook.io/typeorm/docs/migrations

Segment 2

GraphQL- part 1

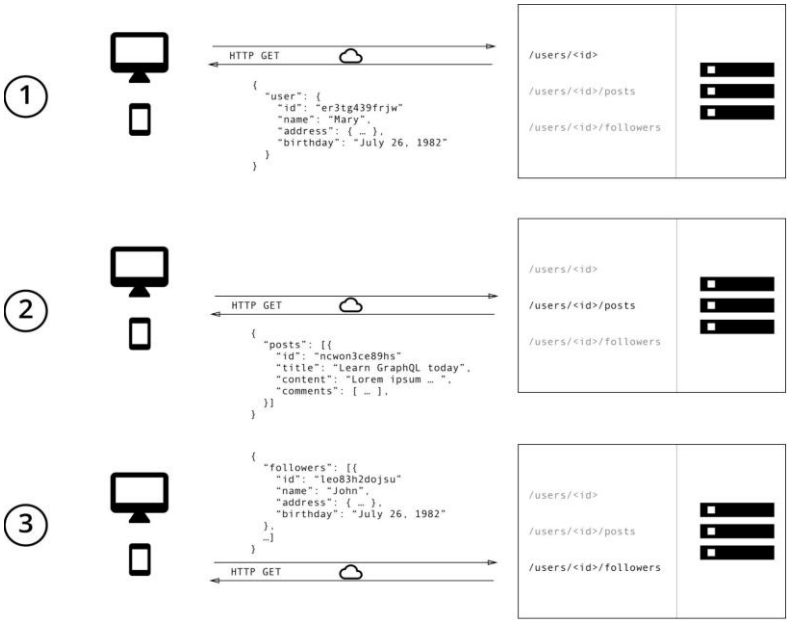


What is GraphQL?

- GraphQL is a query language for APIs and a runtime for fulfilling those queries with your existing data.
- GraphQL is not tied to any specific database or storage engine and is instead backed by your existing code and data.
- It is an *alternative to REST APIs*
- It is not a replacement of REST APIs
- It takes a different approach to REST
- It solves many of the shortcomings that developers experience when interacting with REST APIs.
- Let us look at the differences between GraphQL and REST APIs

GraphQL v/s REST

- This is how REST works.
- With REST, one needs to make *multiple* requests to different endpoints to fetch the required data.
  - In a sense one has to overfetch since the endpoints return additional information that's not needed.



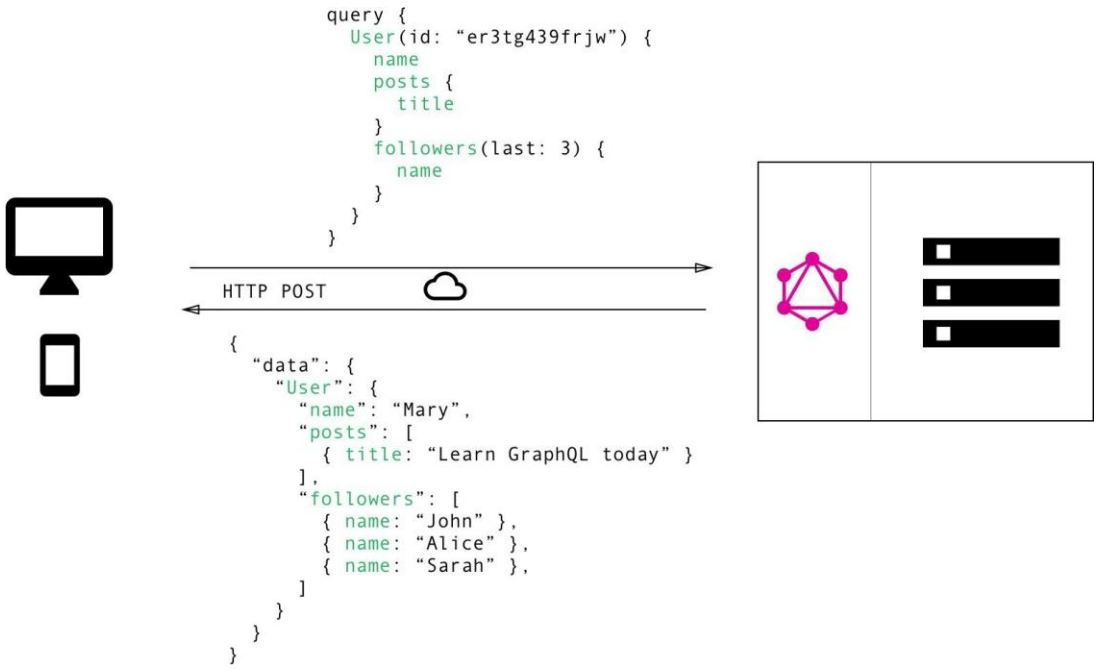
In the example, these could be /users/<id> endpoint to fetch the initial user data.

Secondly, there's likely to be a /users/<id>/posts end point that returns all the posts for a user.

The third endpoint will then be the /users/<id>/follower s that returns a list of followers per user.

GraphQL v/s REST

- You can do the same in GraphQL but with greater efficiency
- You can simply send a single query to the GraphQL server that includes the concrete data requirements. The server then responds with a JSON object where these requirements are fulfilled.



GraphQL v/s REST: *what does that mean?*

- One of the most common problems with REST is that of *over-* and *under fetching*.
  - This happens because the only way for a client to download data is by hitting endpoints that return fixed data structures.
  - It's very difficult to design the API in a way that it's able to provide clients with their exact data needs.
- GraphQL solves this problem-
  - Using GraphQL, the client can specify exactly the data it needs in a query.
  - Notice that the structure of the server's response (*in the previous slide*) follows precisely the nested structure defined in the query.

REST API *downsides*

- A common practice with REST APIs is to structure the endpoints according to the views that you have inside your app.
- The major drawback of this approach is that it doesn't allow for rapid iterations on the frontend.
- With every change that is made to the UI, there is a high risk that now there is more (or less) data required than before.
  - Consequently, the backend needs to be adjusted as well to account for the new data needs.
- With GraphQL this problem is solved.

GraphQL

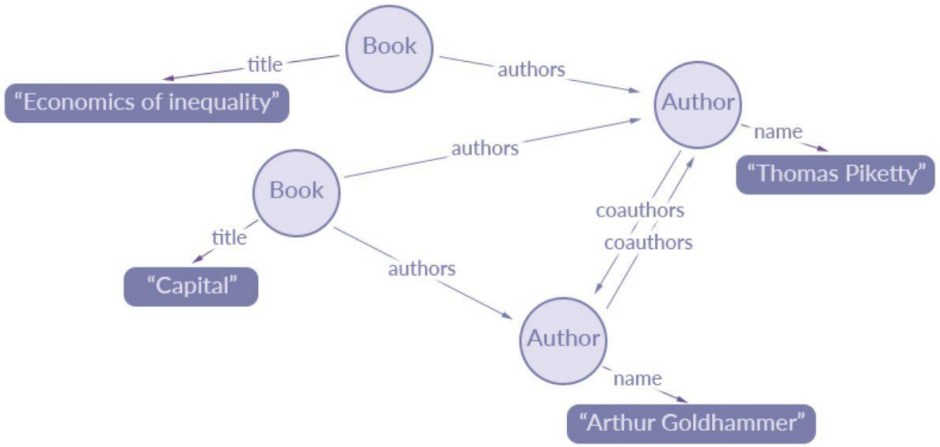
- GraphQL can provide analytics on the backend
  - It allows you to have fine-grained insights about the data that's requested on the backend.
  - As each client specifies exactly what information it's interested in, it is possible to gain a deep understanding of how the available data is being used.
- The biggest advantage offered by GraphQL is
  - Schema & Type System

GraphQL schema and type system

- GraphQL uses a strong type system to define the capabilities of an API.
- All the types that are exposed in an API are written down in a schema using the GraphQL Schema Definition Language (SDL).
- Once the schema is defined, the teams working on frontend and backends can do their work without further communication since they both are aware of the definite structure of the data that's sent over the network.
- Frontend teams can easily test their applications by mocking the required data structures.
- Once the server is ready, the switch can be flipped for the client apps to load the data from the actual API.

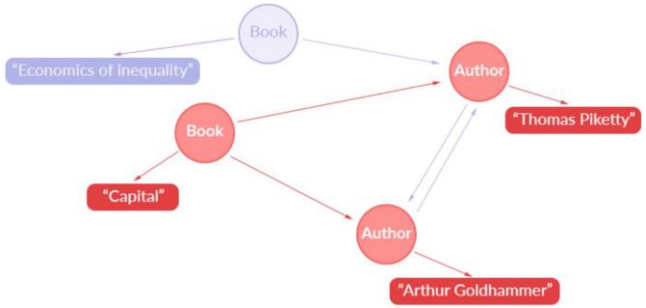
Why is it ”Graph” in GraphQL?

- A lot of data in modern applications can be represented using a graph of nodes and edges, with nodes representing objects and the edges representing relationships between these objects.
- Look at this example: *a library catalogue system*



Reference: https://www.apollographql.com/blog/graphql/basics/the-concepts-of-graphql/

Graph in GraphQL

Query	Outcome
`query { book(isbn: "9780674430006") { title authors { name } } }`	`{ book: { title: “Capital in the Twenty First Century”, authors: [ { name: ‘Thomas Piketty’ }, { name: ‘Arthur Goldhammer’ }, ] } }`
Graph	
	

The “QL” in GraphQL

- Now that we made sense of “Graph”, how are queries written in GraphQL?
- GraphQL uses its own fundamental query language (QL).
- It has its own
  - type system that’s used to define the schema of an API. The syntax for writing schemas is called Schema Definition Language (SDL).
  - Its fundamental QL can be used to construct a query
    - Instead of having multiple endpoints that return fixed data structures, GraphQL APIs typically only expose a *single endpoint*.

GraphQL schema

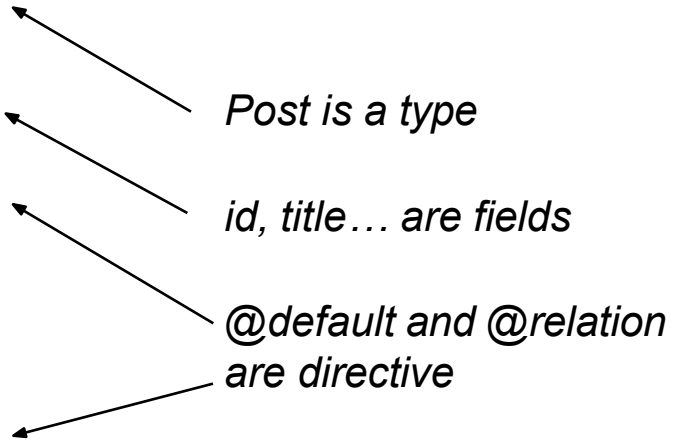
- This is like writing model files
- We will call it *schemas*
- We write them in SDL (Schema Definition Language)
- As an example, if you were building a blog application
- You had two tables- *Post* and *Blog*
- You can use SDL to define them
- In the GraphQL language,
  - we will call them *Post schema* and *Blog schema*

GraphQL Schema

- The main components of a schema definition are the types and their fields.
- Additional information can be provided as custom directives like the @default value specified for the *likes* field.

```
type Post {
  id: String!
  title: String!
  publishedAt: DateTime!
  likes: Int! @default(value: 0)
  blog: Blog @relation(name: "Posts")
}

type Blog {
  id: String!
  name: String!
  description: String
  posts: [Post!]! @relation(name: "Posts")
}
```



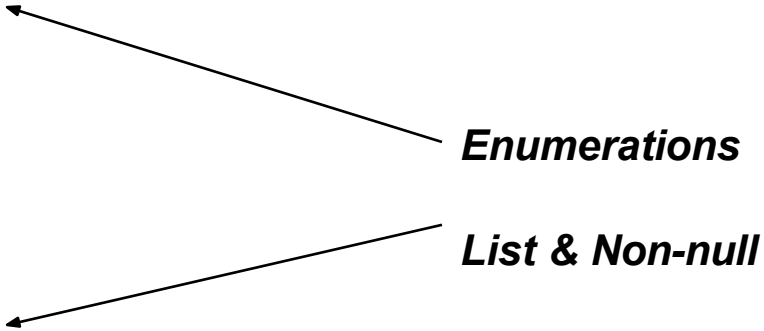
GraphQL schema types

- Here is the best place to learn i.e., its official documentation page
  - https://graphql.org/learn/schema/
- Supported data types:
  - Scalar: Int, Float, String, Boolean, ID
  - Enumeration: Also called *Enums*, enumeration types are a special kind of scalar that is restricted to a particular set of allowed values
  - Lists and Non-Null: When you use the types in other parts of the schema, or in your query variable declarations, you can apply additional type modifiers that affect validation of those values. You can use ! for non-null.
  - Query & Mutation: *it will become clear later on*
  - Union: *it will become clear later on*
  - Input: *it will become clear later on*

GraphQL schema types

```
enum Episode {
  NEWHOPE
  EMPIRE
  JEDI
}

type Character {
  name: String!
  appearsIn: [Episode]!
}
```



GraphQL basic query & lookups

Query Type	Query	Output
Basic	{ status }	{ status: 'available' }
Lookups	{ hero(id: "1000") { id name } }	{ hero: { id: "1000", { name: "Luke Skywalker" } }
Nesting	{ hero { name height } }	{ hero: { name: "Luke Skywalker", height: 1.74 } }
Aliases	{ luke: hero(id: "1000") { name } han: hero(id: "1001") { name } }	{ luke: { name: "Luke Skywalker" }, han: { name: "Han Solo" } }

GraphQL basic query & lookups

Query Type	Query	Output
Lists	{ friends { name } }	{ friends: [{ name: "Luke Skywalker" }, { name: "Han Solo" }, { name: "R2D2" }] }
Multiple types		*Very suitable for searching*

GraphQL Mutations

- **Mutations are just fields that do something when queried.**

```
Query

{ createReview($review) { id } }

Variable(s)

{ review: { stars: 5 } }

Output

{ createReview: { id: 5291 } }
```

example2: Full stack app with GraphQL

- Let us now look at a full example
- So this time
  - Frontend app == React app
  - Backend = Node + Express + GraphQL
  - Database == Cloud MS SQL Server database

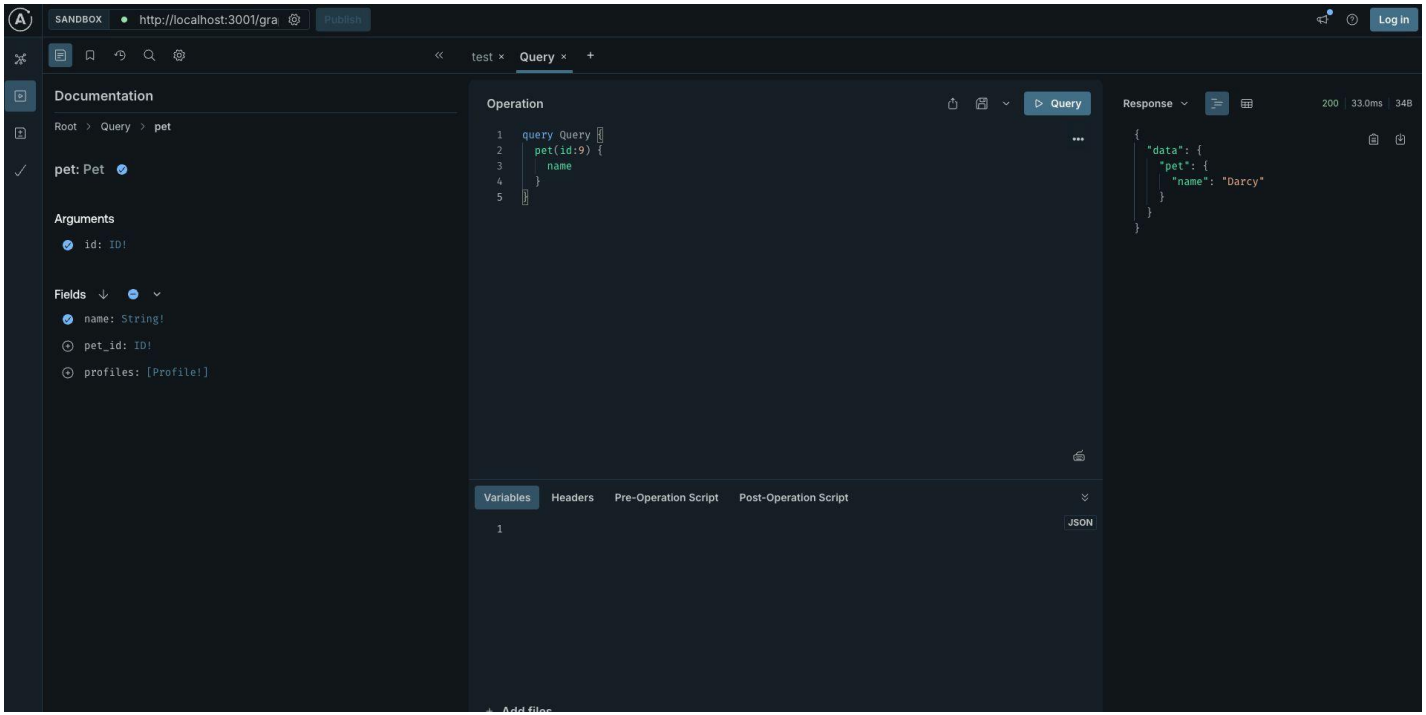
- Let's migrate our Pets app to use GraphQL

example: Node, Express and GraphQL

- Take our existing Example 1 project
- Add the following modules using npm to the project backend
  - graphql, @apollo/server, graphql-tag
- Add our GraphQL and database connectivity code
- We will start the project as `npm run dev` thereby running the API that talks to MS SQL Server database.

example2: Node, Express and GraphQL: *testing API methods*

- **To test the API methods we can use the playground.**
- **Visit localhost:3001/graphql in the provided example**



example2: React app

- After we create the react app
- We will install following modules using npm
  - graphql, @apollo/client
- There are many libraries that you can use in React app to fetch data from GraphQL APIs
  - Apollo family : *very popular*
  - Urql
  - React Query + GraphQL Request
  - …
- In our example
  - We use @apollo/client

example2: React app

- In our example
  - All queries are present inside `src/services/api.ts`
  - GraphQL queries are generally wrapped in the `gql` function.
  - `gql` helps convert query string into a query document

Segment 2

Unit Testing

Unit Testing a Full Stack web app

Feature	Jest	React Testing Library
Role	**Test Runner & Framework**: It finds, executes tests, and provides assertions.	**Utility Library**: It provides tools to render and interact with React components.
Scope	General-purpose JavaScript; works with Node, Vue, Angular, and React.	Specific to React components.
Philosophy	Focuses on correctness, speed (parallelization), and mocking internal logic.	Focuses on user-centric testing; tests should resemble how software is used.
Features	Snapshot testing, code coverage, mocking modules/timers.	Queries for DOM elements (e.g., getByRole, getByText) and event simulation.

Unit testing FSD web applications

- Week 6 and Week 9's labs delved into the unit testing of a front end React web application
- In a full stack web app, it is also possible to unit test the *express* project
- The unit tests would be structured around the database
- Dealing with data from database in unit tests can be a messy process
- There is an approach which can make this process easier- i.e. using the *memory storage*
- In this approach
  - Developer still uses the real data as we store it in persistent storage but we don't need to connect databases because our data will be placed in memory.
  - *HOW?*

Unit testing with memory storage

- In conjunction with express, TypeORM, you can use
  1. SQLite — Small, fast, self-contained, high-reliability, and full-featured SQL engine
- SQLite is great for testing as you don’t need to have an actual database connected
- SQLite database will be stored as a file with `.sqlite3` extension in your project (or we can save completely in memory)
- SQLite is the world's most used database - in every phone, cars etc!

Unit testing with memory storage

A few extra steps:

- Adding Jest and Supertest to our project
- Add sqlite3 module to our project
- Updating the data-source.ts file to use sqlite if running tests

- We will go through a detailed example
  - example 3 

References

- Reference: *The road to react (2025 edition), by Robin Weiruch; Leanpub*
- [https://graphql.org/]
- [https://www.apollographql.com/blog/graphql/basics/ the-concepts-of-graphql/]

Assignment 2 has been online

- Full stack web application
- To be completed in a group
- Worth: 45%
- Requires use of
  - React, (Node, Express) and a Cloud MS SQL database
  - GitHub on a regular basis
  - Consults starting next week

Next week

- GraphQL- *part 2*
- *Security Risks*
- *Documentation in React apps*